

# Evidentia Seamless Collaboration Blueprint

## VS CODE LIVE SHARE REAL-TIME ENVIRONMENT SET-UP

This workspace manual defines the operational sequence required to set up real-time pair programming for the **Evidentia** ecosystem using VS Code Live Share. This approach acts like a real-time collaborative cloud document, avoiding git synchronization conflict overhead during development cycles.

### 1. Extension Installation & Environment Alignment

Before launching a collaborative development stream, both team members (Host and Guest) must instantiate the matching extension subsystem:

1. Launch **VS Code** on your local operating environment.
2. Access the Marketplace Panel by pressing `Ctrl + Shift + X` (or click the 4-box Extensions icon in the Activity Bar).
3. Query for "**Live Share**" and select the official extension bundle distributed by *Microsoft*.
4. Click **Install** and let the internal dependency packages finalize initialization.

### 2. Launching the Collaboration Stream (Host Workflow)

Since the physical project workspace directory is localized on your machine, you will behave as the primary architecture Host:

1. Open your main `evidentia/` root workspace in VS Code.
2. Select the new **Live Share** icon that maps onto your left-side vertical activity ribbon.
3. Click on "**Start Collaboration Session**".
4. Authenticate your session securely using either a **GitHub** account or a **Microsoft / Outlook** credential loop.
5. Once authentication yields access, a unique secure collaboration invite token link is natively generated and auto-copied straight onto your clipboard buffer.
6. Forward this link stream directly to your peer using your communication channels.

### 3. Intercepting the Workspace (Guest Workflow)

Your collaborative peer can hook straight into your running filesystem without downloading repositories, copying databases, or installing local configurations manually:

1. The Guest opens the invite link shared by the Host in any local web browser.
2. When prompted by the system dialog, select "**Open Visual Studio Code**".
3. The Guest signs into their matching GitHub or Microsoft profile when prompted.

4. Within seconds, the complete structural map of the Host's **Evidentia** codebase populates inside the Guest's local editor interface. Editing or saving files writes straight onto the Host machine live.

**Synchronized Interactions:** Both engineers retain autonomous focus states. You can look at separate files independently (e.g., Host works on `App.jsx` while Guest develops `login.php`), or you can double-click your peer's profile card avatar to lock viewports for real-time code reviews.

## 4. Advanced Routing: Exposing Port Boundaries securely

Because Evidentia requires an active React development client, a custom local PHP micro-server gateway, and a MySQL relational process loop, the Guest will need clear visibility into your running system network. Set up port forwarding directly inside the Live Share tab:

### Step 1: Expose the React UI Workspace (Vite Server)

- In your active Live Share management interface tab on the sidebar, locate the **"Shared Servers"** block.
- Click **"Share Server"**.
- Input target application development port: 5173 and label it as React Frontend Gateway.

### Step 2: Expose the Authentication API Standalone Handler

- Click **"Share Server"** a second time.
- Input the designated isolated application target port: 8081 and label it as PHP Backend Server.

**Resulting Behavior:** When the Guest opens their web browser at `http://localhost:5173` on their *own device*, the Live Share sub-layer securely tunnels the data straight from your Windows machine. They can login, click UI components, execute authentications, and test your database seamlessly!

## 5. Shared Terminals & Workspace Controls

If you require your peer to initialize migrations, run test files, or interact directly with terminal components, you can share an integrated command prompt:

- Navigate to the **"Shared Terminals"** line entry in the collaboration panel.
- Click the plus icon to bind a terminal window.
- Assign authorization rights: Choose between **Read-Only** (recommended for inspection loops) or **Read-Write** (allows the Guest to run commands directly inside your Windows console).